AI ASSISTED CODING

ASSIGNMENT-13.5

NAME:P.Sanwitha

ROLL NO:2403A52L09

BATCH-50

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the

code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:

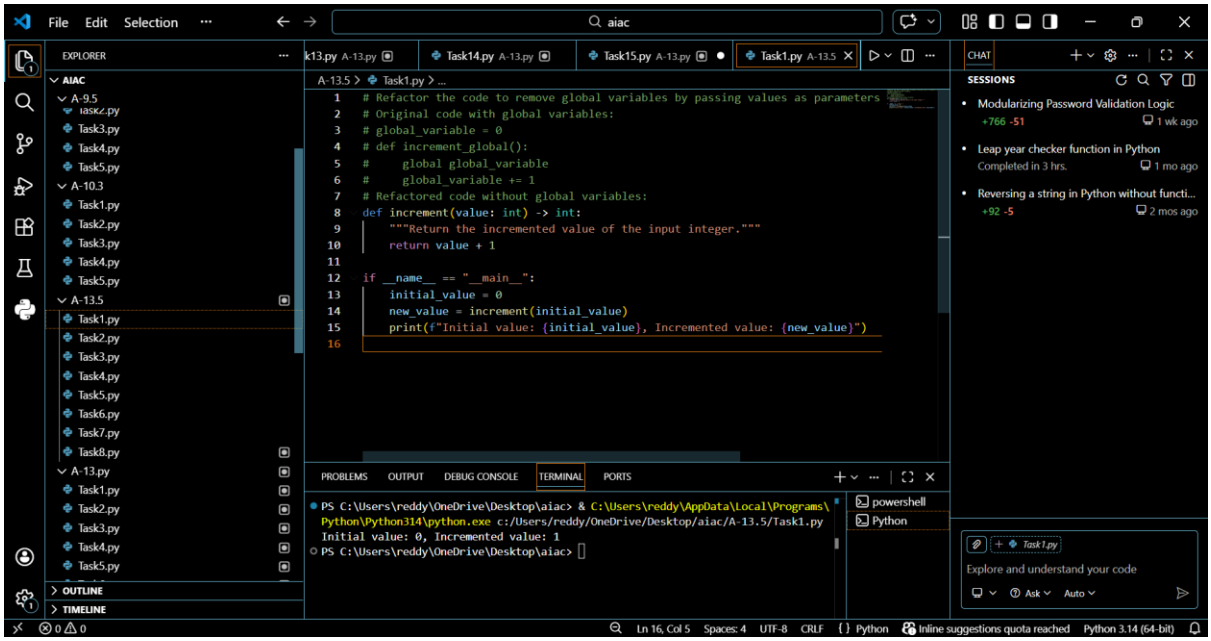
o Refactored version passing rate as a parameter or using a

configuration structure.

PROMPT:

"Refactor the code to remove global variables by passing values as parameters and improving modularity."

CODE AND OUTPUT:



EXPLANATION:

This ensures modularity and testability by avoiding hidden dependencies. Functions become reusable since they rely only on inputs, not external state. It improves clarity and makes unit testing easier.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a

cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

else:

print("Needs Improvement")

Expected Outcome:

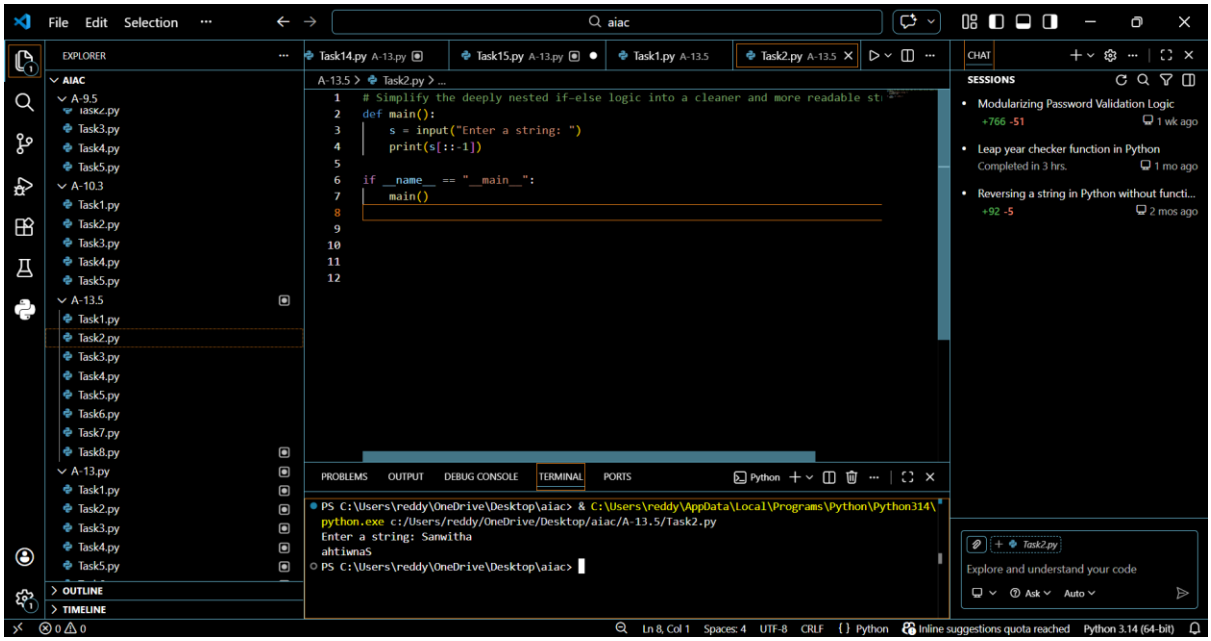
o Flattened logic using guard clauses or a mapping-based

approach.

PROMPT:

"Simplify the deeply nested if–else logic into a cleaner and more readable structure."

CODE and OUTPUT:



EXPLANATION:

Flattening conditionals reduces cognitive load and makes logic easier to follow. Guard clauses exit early, while mappings provide a clean lookup. This improves readability and future scalability.

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()

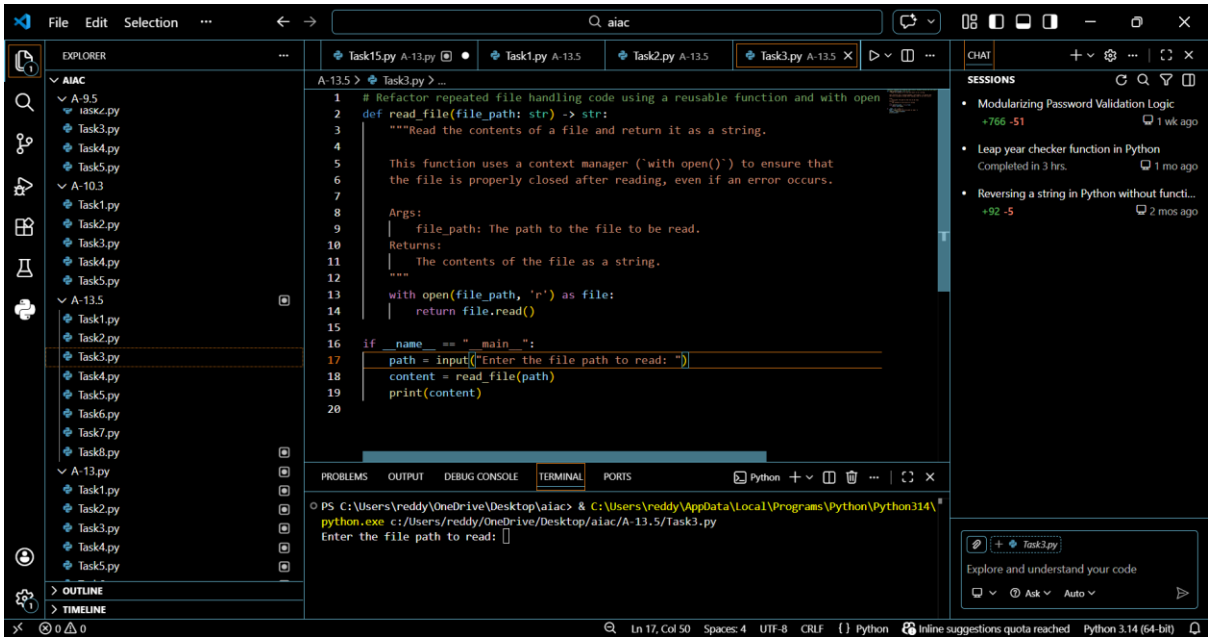
Expected Outcome:

o Reusable function using with open() and parameters.

PROMPT:

"Refactor repeated file handling code using a reusable function and with open() context manager."

CODE AND OUTPUT:



EXPLANATION:

Using with open() ensures safe resource handling and avoids manual closing. Encapsulating file logic in a function reduces duplication. This makes the code concise, reusable, and less error-prone.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• **Focus Areas:**

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

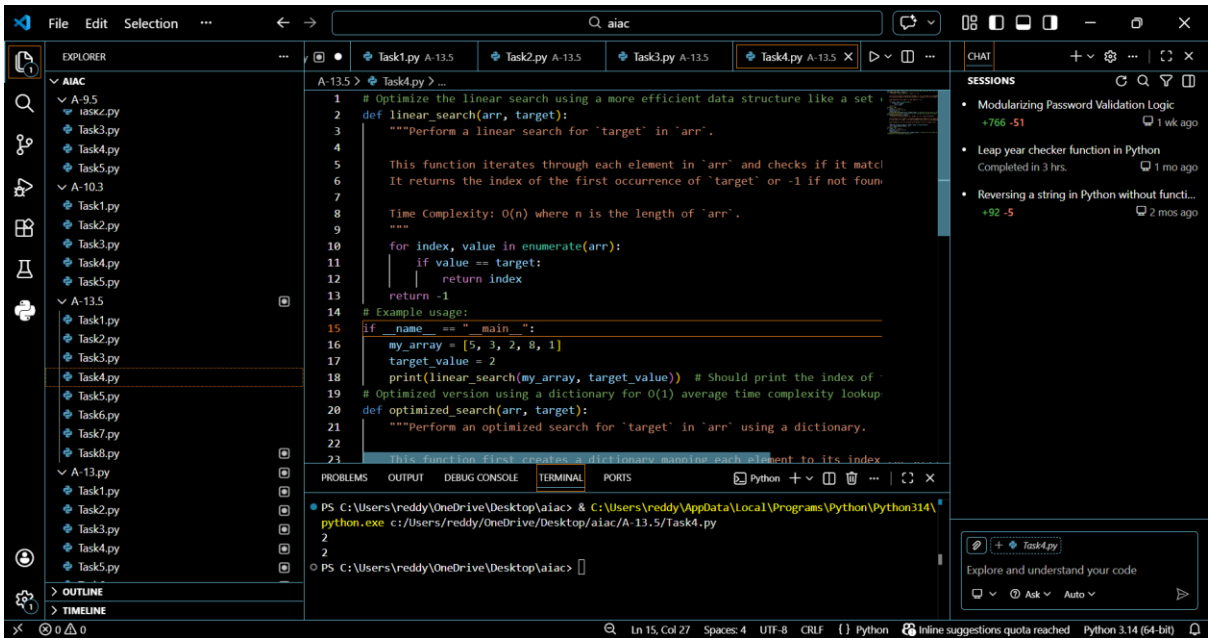
Expected Outcome:

o Use of sets or dictionaries with complexity justification.

PROMPT:

"Optimize the linear search using a more efficient data structure like a set or dictionary."

CODE AND OUTPUT:



EXPLANATION:

Linear search has O(n) complexity, while sets/dictionaries provide average O(1) lookups. This drastically improves performance for large datasets. It also makes the code cleaner and more efficient.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

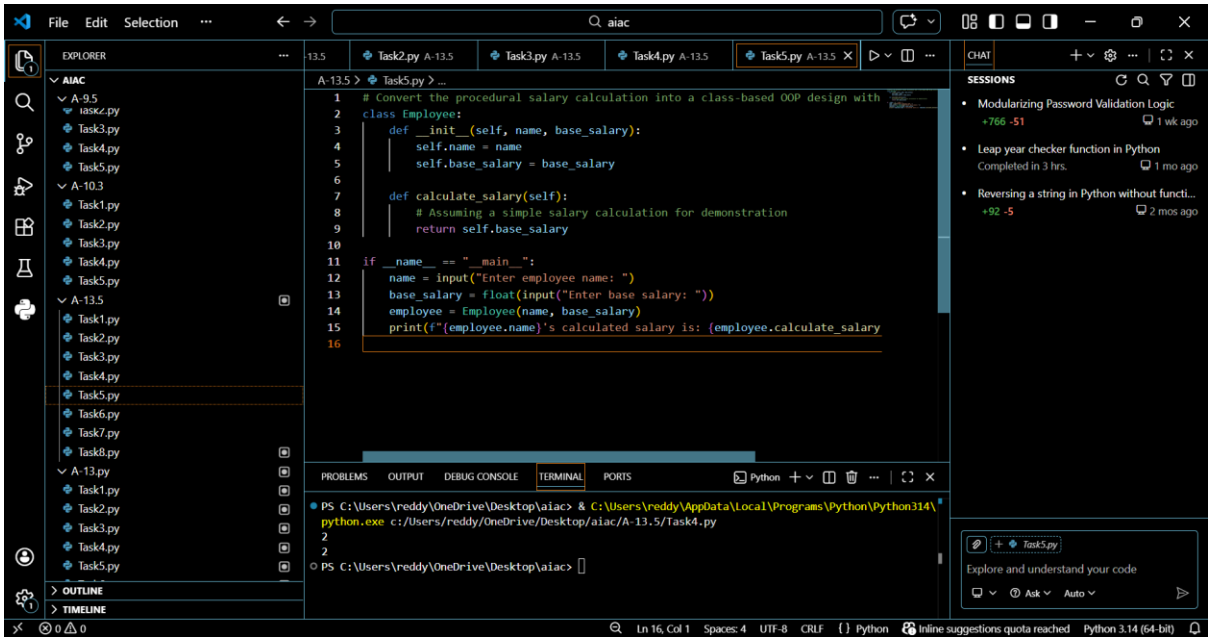
Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

PROMPT:

"Convert the procedural salary calculation into a class-based OOP design with proper encapsulation."

CODE AND OUTPUT:



EXPLANATION:

Object-Oriented design improves structure and reusability. Encapsulation keeps salary, tax, and net logic within a class. This makes the code extensible, testable, and aligned with OOP principles.

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

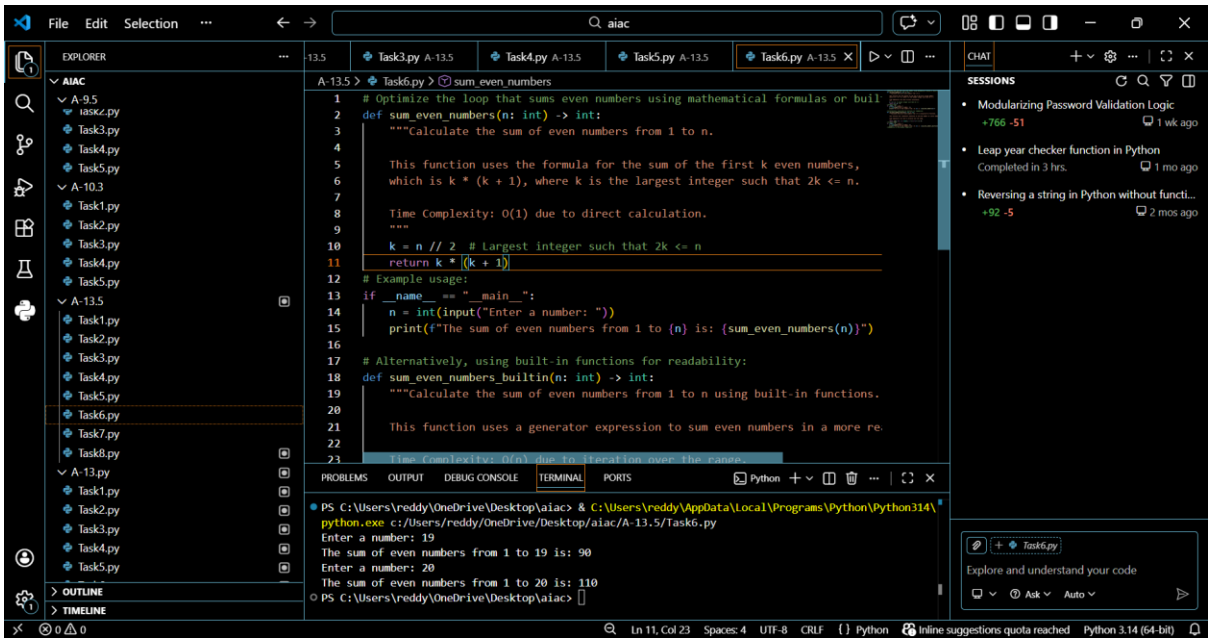
o Optimized logic using mathematical formulas or

comprehensions, with time comparison.

PROMPT:

"Optimize the loop that sums even numbers using mathematical formulas or built-in functions."

CODE AND OUTPUT:



EXPLANATION:

Loops over large ranges are slow; formulas like arithmetic progression or built-ins like sum() with comprehensions are faster. This reduces runtime complexity and improves efficiency significantly.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

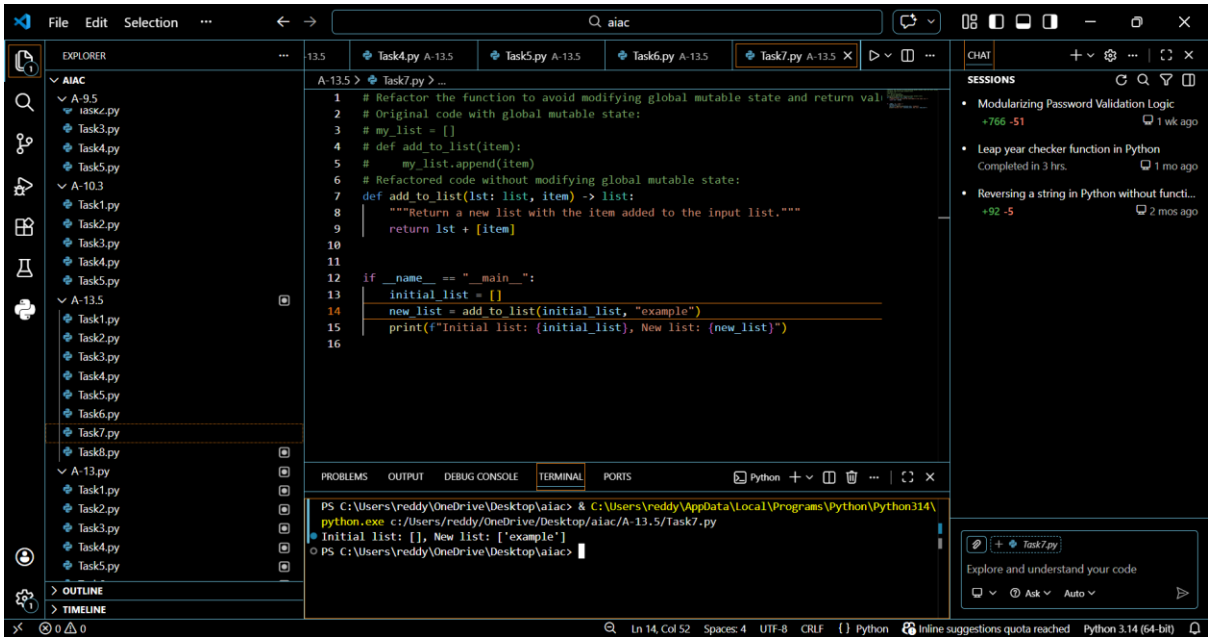
o Refactored function returning values instead of mutating

globals.

PROMPT:

" Refactor the function to avoid modifying global mutable state and return values instead."

CODE AND OUTPUT:



EXPLANATION:

This avoids hidden side effects by eliminating reliance on global or shared mutable data. Returning values makes behavior predictable and easier to test. It improves modularity and ensures functions remain pure, enhancing reliability.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

else:

print("Password too short")

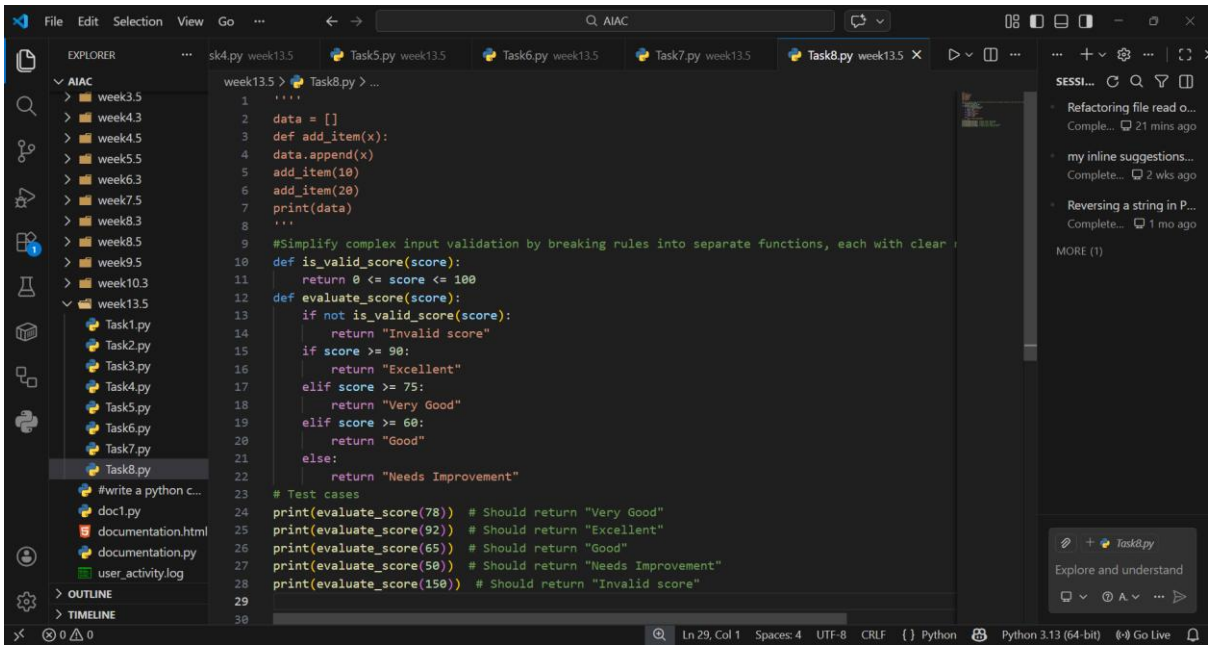
Expected Outcome:

o Separate validation functions with clear responsibility

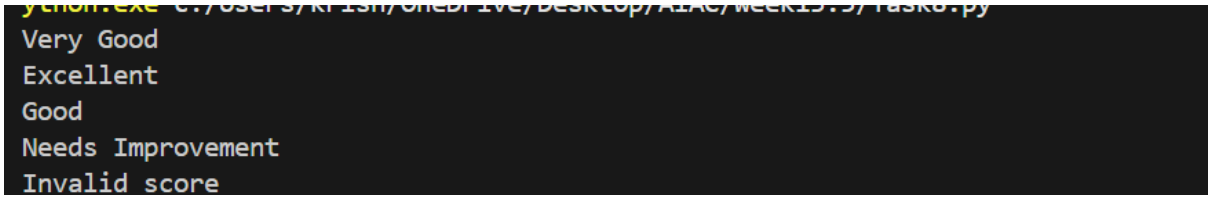
PROMPT:

“Simplify complex input validation by breaking rules into separate functions, each with clear responsibility."

CODE:



OUTPUT:



EXPLANATION:

Splitting validation into modular functions improves readability and testability. Each function handles one rule (length, digit, uppercase), making the logic easier to maintain. This approach supports unit testing and future extensions without nested conditionals.